软件仓库管理：从项目快照到 AI 协作

课程：南京大学《生成式软件工程》2026 年第 3 讲；主讲：蒋炎岩。

视频：https://www.bilibili.com/video/BV1kybV6DE47/，时长 1:40:14。

本文依据公开字幕整理稿与官方讲义重组，面向初学者解释主要概念。本文未逐帧核对全部视频，没有原始时间轴字幕，不提供虚构的精确时间引用。它是有来源说明的学习讲义草稿，不是逐字稿或完成上游视频门禁的成品。下文命令示例是编辑补充，不能当作原课堂操作记录。

学习目标

读完本讲，应能解释项目为什么需要快照，区分工作区、暂存区与历史，判断一次提交的边界，并说明 AI 生成能力提高后为什么仍需要架构设计、验证和版本管理。

1. 为什么只有一个项目目录还不够

软件项目通常以目录的形式出现：代码实现行为，文档表达需求与约定，测试描述可以检查的要求，配置和构建脚本决定如何运行。保留当前目录可以保存当下的工作，却不能自动回答“昨天能运行的版本在哪里”。

复制“作业-v1”“作业-v2”是一种朴素的版本管理方法。它能留下旧文件，但版本关系需要人工记忆。两个人分别修改同一份代码后，文件应该怎样合并？只恢复旧代码却留下新配置，是否仍是一个能运行的项目？这些问题表明，需要管理的是完整项目状态及其变化关系。

Git 将这些状态组织成可回查的历史。它并不要求先使用 GitHub：本地仓库负责记录版本，GitHub 等平台负责托管以及围绕 issue、审查和发布进行协作。

本章小结：版本管理让项目保留可理解的过去，为恢复、比较和协作提供基础。

2. 快照如何构成历史

Git 在逻辑上记录项目快照。快照描述被跟踪文件在某个时刻的内容与目录位置；提交进一步关联说明、作者和父提交。沿父提交回溯，就能看到项目如何演变。

完整快照不意味着每次物理复制所有文件。相同内容可以复用，目录对象引用已有内容，因此少量修改不需要重复保存所有未变的部分。理解这个模型，可以避免把 Git 仅看成一串压缩包。

概念	承担的作用
blob	保存文件内容的字节序列
tree	组织目录条目、名称及其引用的对象
commit	关联根目录树、父提交和提交信息

概念	承担的作用
分支	指向某个提交的可移动引用
HEAD	通常标识当前分支，也可以直接指向提交

课堂使用可视化工具观察真实仓库对象。这种学习方法的价值在于把“对象”“引用”等抽象名词与磁盘中的实际数据联系起来，观察一次操作究竟改变了什么。

本章小结：Git 保存项目状态，也保存状态之间的关系；对象复用使快照可以高效存储。

3. 为什么要区分工作区和暂存区

工作区是正在编辑的文件，暂存区保存下一次提交准备采用的内容，提交历史保存已经记录的快照。暂存区允许开发者组织一次修改，而不是把所有正在探索的内容一次打包。

编辑补充示例：

```
git init

git add hello.c

git diff --cached

git commit -m "feat: add a minimal hello program"
```

如果执行 `git add hello.c` 后又修改了文件，普通 `git commit` 提交的是已暂存的版本。后续编辑仍留在工作区，需要再次暂存才进入这次提交。`git diff` 用于查看未暂存的修改，`git diff --cached` 用于查看已暂存的修改。

这也说明为什么不应不经检查就执行 `git add .`：它可能把编译产物、临时文件和私人配置一起纳入提交。命令合法，不代表项目选择合理。

本章小结：提交是经过选择和组织的变更；暂存区提供了表达这一选择的位置。

4. 分支、合并与冲突解决了什么问题

分支让不同工作可以从共同起点向前推进。一个人修改界面，另一个人修改解析器，双方先记录各自的变化，再合并结果。

Git 可以依据共同祖先与双方修改计算合并。当同一段文本出现无法自动决定的变化时，工作区通常会出现冲突标记，展示双方内容。开发者需要理解上下文，把文件改成最终需要的结果，再标记解决并完成提交。

合并成功只说明文本变化得到组合，不说明行为正确。两个分支即使修改不同文件，也可能在接口、配置或业务规则上不兼容，所以还要验证组合后的项目。

官方讲义补充说明了 rebase：它把一串改动重新应用到新的起点，通常会产生新的提交标识。是否可以整理历史取决于协作约定；别人已经依赖的提交不宜随意改写。

本章小结：分支隔离尝试，合并汇合变化；验证负责判断组合后的行为。

5. 怎样组织一次有意义的提交

课程最值得记住的工程原则是，把项目看作由小而完整的变更推进的一系列快照。这里的“小”首先指目的清楚，而非行数少。实现一个功能可能涉及代码、测试和文档，它们共同服务同一意图，可以组成一次完整提交。

相反，修复错误时顺手重排整个项目，会混合不同目的。将来出现故障，读者难以判断哪些变化相关，也难以单独撤销。保持提交边界清楚，可以降低审查、恢复与定位问题的成本。

提交说明应补充 diff 没有直接表达的意图。只有 `update` 或“做了些东西”，无法帮助后来的人理解变化。标题概括行为，正文解释动机、约束和取舍；具体格式服从项目约定。

编辑补充示例：

```
fix(parser): accept repeated spaces between arguments


Ignore extra spaces between arguments while preserving spaces inside quotes.
```

Conventional Commits 提供 `type(scope): description` 的一种组织方式，但格式正确不等于说明充分。代码能否构建、相关测试能否通过，同样影响这次提交能否成为可靠的历史检查点。

本章小结：好的提交边界让变化可以解释，好的说明让后来的人知道为什么要保留它。

6. 什么应该进入仓库

仓库保存共同维护的项目状态，不必保存工作目录里出现的一切。编译产物和日志通常能由源码及工具重新生成，不适合每次提交；机器绝对路径和个人编辑器偏好也可能让他人无法复现。

文件类型	判断依据
源码、测试、构建说明	通常是共同维护的项目内容
编译产物、运行日志	通常忽略，防止膨胀及无意义冲突
私人路径和临时文件	通常排除，避免依赖个人机器
凭据与 API key	不进入版本历史
依赖锁文件或生成头文件	依据复现需求、工具链和团队约定决定

“生成文件一律不提交”过于简单。锁文件可以固定依赖版本，某些生成头文件可以减少使用者安装额外工具链的负担。关键问题是为什么共享它，以及怎样保持一致。

官方讲义提醒：`.gitignore` 不会让已经跟踪的文件自动退出历史。误提交密钥后，仅删掉当前文件不能消除旧版本中的泄露，应按凭据类型处理失效、轮换及历史清理。

本章小结：根据文件在项目中的角色决定是否提交，不按文件名机械判断。

7. 历史如何帮助验证与恢复

及时保存合理的检查点，可以让失败的探索有退路。`git bisect` 利用好版本与坏版本间的检查结果缩小范围，帮助找到引入问题的提交；`git blame` 帮助追溯代码行的来源；`git revert` 则以新提交表达对既有变化的撤销。

这些工具的效果取决于历史质量。如果一个提交混合了许多独立目的，定位到它后仍需拆解；如果大量中间版本无法构建，也会增加二分检查的困难。

GitHub 上的 issue、PR 和检查结果属于平台协作信息，并不自动包含在 Git 的文件历史中。课堂提到的 `gh`、`glab` 可以让 Agent 通过命令行连接这些协作步骤。使用平台工具仍要区分本地验证、远端推送、提出合并申请和正式合入。

本章小结：历史不仅用来存档，也可以成为定位、审查和恢复的工具。

8. AI 能写代码以后，人还要做什么

课程建议在架构阶段先集中讨论：明确职责、接口和约束，减少相互影响的共享状态，再把边界写成文档、规则与示例。接口稳定后，更多实现细节可以交给 Agent 独立处理。

课堂的编程可视化项目说明了这种做法。通过清晰接口和较少的核心共享状态，组件可以成为相对独立的黑盒，Agent 只需要理解接口，而不必读取所有实现。

实现阶段可以安排多个 Agent 处理相互独立的任务，但并发数量并非质量保证。任务依赖、文件覆盖、接口兼容和集成责任仍需要管理。讲师展示的是个人工作方式，不是所有项目都应遵守的并发上限。

有界面的项目应尽早运行并观察效果。只读“已完成”不能发现按钮位置、交互顺序或文字密度的问题；持续反馈可以尽早纠正设计偏差。

课程结尾提出进一步的问题：当生成速度远超人的理解和验收速度，版本控制与协作方式会遇到什么瓶颈？这节课提出问题，为后续讨论铺垫，不把设想写成已实现的自动化系统。

本章小结：人的贡献更多体现在定义目标、形成设计边界和判断结果，版本管理让探索过程可控。

9. 怎样借助 AI 学习工程知识

课程反复强调注意力：看到成熟项目时，留意目录、文档和操作反馈。先想自己会如何解决，再比较现成方案；如果对方更好，追问它解决了哪个自己忽略的约束。

这种方法被称为 virtually implement。它适用于读代码、论文、README，也适用于检查 AI 生成的解释。重点是形成预测并用证据修正理解。

学习 Git 时，可以要求 Agent 在操作前解释将影响哪些文件、暂存内容与历史引用，在操作后展示状态变化。这样每次真实任务都能补充一个概念，而不只是记住一条命令。

小结与练习

项目的演变应能被分解为可理解的变更。快照提供可恢复状态，分支和合并组织协作，清楚的提交边界与验证让历史真正有用。

练习：在一次性练习仓库中，修改文件后先暂存，再修改一次，比较两个 diff；创建两条修改同一行的分支，观察冲突；检查自己的项目中是否存在不应跟踪的产物。让 Agent 先解释预期结果，再执行并核对。

来源与许可

- 原课程：蒋炎岩，南京大学《生成式软件工程》2026 年第 3 讲，视频 https://www.bilibili.com/video/BV1kybV6DE47/。
- 官方讲义《版本管理》。
- 公开字幕整理稿，来源仓库尚未将本讲标记为完成人工校对。
- 原课程与字幕整理来源采用 CC BY-NC 4.0。本文对相关内容作删减、重排和解释，保留署名并按相同条件提供对应改编内容，不代表讲师或学校认可。

本文未加入未经核实的视频截图，未复现课堂工具，也未验证讲师对模型能力的个人比较。后续获得原始 SRT 和画面后，应补做时间轴与图文核验。